“Parallel File Search Using Threads”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

VENKATA SAI KRISHNA	**2520030027**
PUJITH PAVAN KUMAR	**2520030029**
AJAI MANIKANTA	**2520030320**
BANDLA VINAY	**2520030437**

Under the Esteemed Guidance of

Dr. Muniraju Naidu V
Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled **"Parallel File Search Using Threads"** is a record of Bonafide work of **VENKATA SAI KRISHNA - 2520030027, PUJITH PAVAN KUMAR - 2520030029, AJAI MANIKANTA - 2520030320, BANDLA VINAY - 2520030437** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

VENKATA SAI KRISHNA	**2520030027**
PUJITH PAVAN KUMAR	**2520030029**
AJAI MANIKANTA	**2520030320**
BANDLA VINAY	**2520030437**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Parallel File Search Using Threads"** is a bonafide work done and submitted by **VENKATA SAI KRISHNA – 2520030027, PUJITH PAVAN KUMAR – 2520030029, AJAI MANIKANTA – 2520030320, BANDLA VINAY – 2520030437** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance and support of many people. We take this opportunity to express our sincere thanks to everyone who helped us, directly or indirectly, during the course of this work.

We are greatly thankful to our Course Coordinator Dr. K. Anusha, and our project guide

Dr Muniraju Naidu Department of Computer Science and Engineering, for their constant encouragement, valuable suggestions and guidance throughout the Operating Systems and Systems Programming course project.

We express our gratitude to Dr. P Venkateshwara Roa (CSE), Head of the Department of Computer Science and Engineering, for providing the facilities and the academic environment required to complete this project.

We thank all the teaching and non-teaching staff members of the department who supported us during this work. Finally, we sincerely thank our parents, friends and classmates for their help and cooperation.

VENKATA SAI KRISHNA	**2520030027**
PUJITH PAVAN KUMAR	**2520030029**
AJAI MANIKANTA	**2520030320**
BANDLA VINAY	**2520030437**

Abstract

Searching for files and for text inside files is a common task, but when a directory structure is large and its files are examined one after another, the search can take a long time and leave the processor cores of a modern computer idle. For students learning Operating Systems, it is also not always clear how threads, synchronization and file-system concepts work together in a real program.

This project presents Parallel File Search Using Threads, a menu-driven terminal application developed in C11 using POSIX threads (pthreads) on Ubuntu/Linux, including Ubuntu under the Windows Subsystem for Linux (WSL). The application uses a master-worker thread pool with a shared, dynamic task queue (producer-consumer pattern). Worker threads traverse directories and examine files concurrently; mutexes (pthread_mutex_t) protect shared data and condition variables (pthread_cond_t) coordinate the workers without busy-waiting. Circular symbolic links are prevented by tracking the device and inode numbers of visited directories.

Its ten menu options search by filename (case-insensitive substring and shell wildcards), search text inside ASCII/UTF-8 files while skipping binary files, search by file extension, search the Windows PC through WSL mount points, search by file size, and search hidden and configuration files. The application also finds duplicate files using file-size matching, a 64-bit FNV-1a hash and byte-by-byte verification, benchmarks the search with 1, 2, 4, 8 and 12 threads and calculates the speedup T1 / TN, and records every search in a history log. Each search ends with a summary of the files scanned, matches found, threads used and time elapsed.

The project demonstrates multithreading, concurrent execution, thread synchronization, file handling, directory traversal and resource management. The benefit of multithreading depends on the workload, the storage device and caching, and the benchmark results in this report show that more threads do not always give a faster search. The tool is a lightweight educational application and is not intended to replace professional search utilities.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Declaration		i
	Certificate		ii
	Acknowledgement		iii
	Abstract		iv
1	Introduction		1–3
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	2
		1.5 Importance of the Application	3
		1.6 Target Users / Audience	3
2	OS Concepts and Background Study		4–8
		2.1 Operating System Concepts Related to the Project	4
		2.2 Existing Methods / Approaches	6
		2.3 Proposed Approach	7
		2.4 Advantages of the Proposed Approach	7
		2.5 CO-wise Mapping	8
3	System Requirements		9–10
		3.1 Hardware Requirements	9
		3.2 Software Requirements	9
		3.3 Operating System Environment — Ubuntu / WSL	9
		3.4 Development Tools and Compilers	10
4	Technology and Tools Used		11–13
		4.1 C Programming	11
		4.2 GCC Compiler	11
		4.3 POSIX Threads and Linux APIs	12
		4.4 WSL Host Interoperability	12
		4.5 Linux Commands and Utilities	12
		4.6 Other Tools	13
5	System Architecture and Design		14–18
		5.1 Overall System Architecture	14

S. No.	Chapter	Topics / Subtopics	Page No.
		5.2 System Workflow	15
		5.3 Flowchart / Data Flow	16
		5.4 Module Description	16
		5.5 Input and Output Design	17
6	Implementation		19–26
		6.1 Module-wise Implementation	19
		6.2 Algorithm / Pseudocode	23
		6.3 Core C Implementation	24
		6.4 Linux System Calls / APIs Used	25
		6.5 Process / Thread Implementation	25
		6.6 IPC / Synchronization Implementation	25
		6.7 Memory Management Implementation	26
		6.8 User Interface Implementation	26
7	Execution and Demonstration		27–36
		7.1 Compilation in Ubuntu	27
		7.2 Execution Commands	27
		7.3 Main Menu	27
		7.4 Search by Filename	28
		7.5 Search Text Inside Files	29
		7.6 Search by File Extension	30
		7.7 Search Windows PC	30
		7.8 Search by File Size	31
		7.9 Search Hidden / Config Files	32
		7.10 Find Duplicate Files	32
		7.11 Benchmark Performance	33
		7.12 Search History Log	34
		7.13 Testing	35
		7.14 Demonstration of OS Concepts	36
8	Conclusion		37–39
		8.1 Summary of the Project	37
		8.2 Objectives Achieved	37
		8.3 Results Achieved	38
		8.4 Skills and Learning Outcomes	38
		8.5 Limitations	38
		8.6 Future Scope	39
9	References		40–41

S. No.	Chapter	Topics / Subtopics	Page No.
		9.1 Books	**40**
		9.2 Research Papers	**40**
		9.3 Linux / GNU Documentation	**40**
		9.4 Online Resources	**41**

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

Searching is one of the most frequent tasks that users and programmers perform on a computer. A modern machine can hold hundreds of thousands of files spread over deeply nested folders, and a user often needs to find a particular file by name, find every file of a certain type, or locate a word or code fragment inside many files. Doing this by hand is slow, so the operating system and its utilities provide search tools.

Traditional search tools usually visit files one after another. This is simple, but the processor of a present-day computer has several cores, and a sequential search leaves most of them idle while the program waits for the disk or examines one file at a time. An operating system offers threads so that one program can do several independent pieces of work at the same time inside a single process.

Linux is well suited for studying this idea. It provides the POSIX threads (pthreads) library for creating threads, mutexes and condition variables for coordinating them, and file-system interfaces such as `opendir()`, `readdir()`, `stat()` and `lstat()` for walking directory trees. This project, **Parallel File Search Using Threads**, combines these interfaces in a menu-driven C application that distributes search work among a pool of worker threads.

1.2 Problem Statement

Searching for a file or a text pattern across a large directory structure can be time-consuming when files are examined sequentially. Searching for text inside files requires every file to be opened and read, and finding duplicate files or filtering files by size or type adds further work. Doing this manually, or with a program that uses a single thread, is inconvenient and under-uses the available processor cores.

The problem is to develop a multithreaded C application that searches files and directories concurrently, supports filename, content, extension, size, hidden-file and duplicate searches, shares data between threads safely, handles filesystem problems such as unreadable folders and circular symbolic links, and presents the results and search statistics in a clear form. The project does not claim that parallel searching is always faster; its performance depends on the workload, the storage device and caching, and the application therefore includes a benchmark to measure it.

1.3 Scope of the Project

The project is a menu-driven terminal application written in C11 for Linux/Ubuntu, including Ubuntu running under the Windows Subsystem for Linux (WSL). Its scope covers the ten options of the main menu:

- Search by filename using case-insensitive substring matching or shell wildcards (Option 1).
- Search for text inside ASCII/UTF-8 files, skipping binary files (Option 2).
- Search by file extension (Option 3) and search the Windows PC through WSL mount points (Option 4).
- Search by file size, larger or smaller than a threshold (Option 5), and search hidden and configuration files (Option 6).
- Find duplicate files using size matching, a 64-bit FNV-1a hash and byte-by-byte verification (Option 7).
- Benchmark the search with 1, 2, 4, 8 and 12 threads and compute the speedup (Option 8).
- View and clear the search history log (Option 9) and exit the program (Option 10).

The following limitations define the boundaries of the project. The application is a text interface only; there is no graphical dashboard. It depends on POSIX and Linux interfaces and is not intended to run natively on Windows. Searches are case-insensitive, and regular expressions are not supported. Inter-process communication between separate processes is not implemented, because all workers are threads of one process. Performance results depend on the machine, the folder searched and caching (Section 7.11).

1.4 Objectives of the Project

The objectives of the project are:

1. To develop a file-searching application using the C programming language (C11).
2. To implement multithreading using POSIX threads and a master-worker thread pool.
3. To support filename searches using substrings and shell wildcard patterns.
4. To support searching for text inside files, with detection and skipping of binary files.
5. To support metadata-based searches by extension, size and hidden-file status, and to search Windows files through WSL.
6. To traverse directories and subdirectories recursively and distribute the work dynamically among worker threads.

7. To protect shared data using mutexes and coordinate workers using condition variables.
8. To handle filesystem errors safely, including permission errors and circular symbolic links.
9. To detect duplicate files using file-size filtering, hashing and byte-by-byte comparison.
10. To measure performance for different thread counts and to keep a record of previous searches in a history log.
11. To understand and demonstrate practical operating system concepts through a working program.

1.5 Importance of the Application

For Operating Systems students, the application shows how abstract concepts appear in a running program. The thread pool shows how threads are created and how work is shared; the task queue shows the producer-consumer problem; the mutexes and condition variables show how race conditions are avoided; and the inode table shows how the file system identifies files uniquely so that circular symbolic links can be detected.

For programmers and general users, the tool gives a quick way to locate files, find text in a code base, find large files for disk clean-up, list hidden configuration files and detect redundant copies of files. The benchmark and history features allow the effect of the thread count to be observed experimentally, which also shows that adding threads does not always make a search faster.

1.6 Target Users / Audience

- Students who are studying Operating Systems and Systems Programming and want a working multithreaded reference program.
- Developers who need to search source code, configuration files and logs across large folders.
- Users who want to find large or duplicate files for disk clean-up.
- Windows users who run Ubuntu under WSL and want to search their Windows folders from the Linux terminal.
- Learners of POSIX thread programming who want to study thread pools, mutexes and condition variables.

CHAPTER 2
OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

This section describes the operating system concepts on which the project is based. For each concept, the last sentence explains how it appears in the project.

2.1.1 Operating System Services and System Calls

An operating system offers services such as program execution, file-system access and resource allocation. A user program requests them through system calls, which in C are usually used through library wrapper functions. This project uses directory and metadata calls (`opendir()`, `readdir()`, `stat()`, `lstat()`) and file reading functions to examine the file system.

2.1.2 Processes and Threads

A process is a program in execution with its own address space. A thread is a unit of execution inside a process; the threads of one process share its memory, open files and data, but each has its own stack and registers. Creating a thread is cheaper than creating a process, and shared memory makes it easy to hand work from one thread to another. This project runs as one process that creates several worker threads.

2.1.3 Multithreading and Concurrent Execution

When a computer has several processor cores, threads of one process can run at the same time, and while one thread waits for the disk another can use the processor. Performance does not always improve with more threads, because creating threads, waiting for locks and competing for the storage device add overhead. The project's benchmark option measures this effect (Section 7.11).

2.1.4 Master-Worker Thread Pool

In a thread pool a fixed number of worker threads are created once and repeatedly take tasks from a shared queue. A master part of the program prepares the first tasks, starts the workers and waits for them to finish. Because workers take new tasks whenever they become free, the load is balanced dynamically; a worker that finishes a small folder immediately takes another task. The project uses this architecture.

2.1.5 Producer-Consumer Problem and the Task Queue

In the producer-consumer model some threads produce items and others consume them through a shared buffer. In this project the workers are both consumers and producers: they take a directory or file task from the queue, and when the task is a directory they add its entries as new tasks. The queue must therefore be protected, and consumers must wait when it is empty.

2.1.6 Race Conditions and Mutex Locks

A race condition occurs when two threads update shared data at the same time and the result depends on timing; for example, two threads incrementing the same file counter can lose an update. A mutex (mutual exclusion lock, `pthread_mutex_t`) allows only one thread at a time to enter a critical section. The project uses mutexes to protect the task queue, the search statistics and the output stream.

2.1.7 Condition Variables

A condition variable (`pthread_cond_t`) lets a thread sleep until another thread signals that a condition has become true, which avoids busy-waiting, where a thread repeatedly checks a variable and wastes processor time. The project uses two condition variables: `has_work`, which wakes workers when tasks are available, and `all_done`, which tells the master that all work has finished.

2.1.8 File System, Directories and Metadata

A file system organises data into files and directories. A directory is read entry by entry with `opendir()` and `readdir()`, and the metadata of each entry (size, type, device and inode number) is obtained with `stat()` or `lstat()`. The project reads these to match names, filter sizes and identify directories.

2.1.9 Inodes and Symbolic Links

An inode stores the metadata of a file, and the pair of device number (`st_dev`) and inode number (`st_ino`) identifies a file or directory uniquely. A symbolic link is a file that refers to another path; `lstat()` describes the link itself, while `stat()` follows it. A link that points back to a parent folder creates a cycle that would make a recursive search run forever. The project records the device and inode of the directories it visits in an inode hash table and does not enter a directory twice.

2.1.10 Resource Management

Resource management is the task of using limited resources such as CPU cores and I/O bandwidth efficiently. The project chooses the number of worker threads according to the number of hardware cores (12 in the test system, Figure 7.2), allows the thread count to be varied in the benchmark, and records every search with its runtime in a log.

2.2 Existing Methods / Approaches

Linux already provides well-established tools for searching. The most common are described below.

- `find` walks a directory tree and tests names, types, sizes and times; it processes entries one after another.
- `grep -r` searches the contents of files recursively and prints matching lines.
- `ls -a` and `ls -R` list hidden files and directory contents; `du` reports disk usage.
- Graphical file managers and desktop search tools provide a search box for names and sometimes contents.
- Duplicate-finder utilities compare files by size and checksum.

These tools are mature and suitable for professional use. The project does not replace them; it combines several search operations in one menu-driven C program whose multithreaded design is visible in the source code. Table 2.1 compares typical sequential tools with the proposed application.

Table 2.1: Comparison of a typical sequential search approach and the proposed application

Feature	Typical sequential approach	This project
Processing of files	One after another in a single thread	Distributed among worker threads
Load balancing	Not applicable	Dynamic: workers take tasks from a shared queue
Filename search	Yes	Yes: substring and wildcard patterns (Option 1)
Content search	Yes (separate command)	Yes, with binary-file detection (Option 2)
Extension / size / hidden filters	Separate options of different commands	Options 3, 5 and 6 in one menu
Duplicate files	Separate utility	Size, FNV-1a hash and byte comparison (Option 7)
Windows files from Linux	Manual path handling	WSL mount points and quick targets (Option 4)
Symbolic-link cycles	Depends on options used	Detected using inode tracking
Performance measurement	Manual timing	Built-in benchmark and history log (Options 8, 9)

2.3 Proposed Approach

The proposed system is a C program for Linux/Ubuntu with a menu-driven terminal interface. When the user selects an option, the dashboard in `main.c` collects the folder, the search term and any other parameters, and calls the search orchestrator `run_search()`. The orchestrator places the first tasks in a master task queue, starts the worker threads and waits until they have finished. Each worker repeatedly takes a task, reads a directory or examines a file, adds subdirectories to the queue, and updates the shared results and statistics under mutex protection. Before a directory is entered, its device and inode numbers are checked in the inode hash table to prevent circular symbolic-link traversal.

When all work is finished, the matching results are shown with a search summary (files scanned, matches found, threads used, time elapsed and scanning speed), and the search is recorded in the file `search_history.txt`. The program needs no external library other than the standard C library and POSIX threads.

2.4 Advantages of the Proposed Approach

- Concurrent processing: independent search tasks are shared among several worker threads.
- Dynamic load balancing: workers take new tasks as soon as they are free.
- Several search modes in one interface: filename, content, extension, size, hidden files, Windows PC and duplicates.
- Safe shared data: mutexes protect the queue, statistics and output; condition variables avoid busy-waiting.
- Safe traversal: permission errors are handled and symbolic-link cycles are detected using inodes.
- Built-in measurement: the benchmark and the history log allow the effect of the thread count to be studied.
- Educational value: the program demonstrates threads, synchronization and file-system concepts in one project.
- Easy to run: a single terminal program with default values for most prompts.

2.5 CO-wise Mapping

Table 2.2 maps the project to the Course Outcomes (CO1 to CO6) of the Operating Systems and Systems Programming course. The mapping lists only features that are present in the application. Inter-process communication between separate processes is not implemented; CO3 is mapped to the thread-level communication and synchronization that the project does implement.

Table 2.2: Mapping of the project to the Course Outcomes

CO	Course Outcome area	Implementation in the project	Status
CO1	Operating System services / system calls	opendir(), readdir(), stat(), lstat() and file reading; Linux and WSL interfaces	Implemented
CO2	Process management	One process with a pool of POSIX worker threads created for each search; thread count configured for the benchmark	Implemented
CO3	Inter-Process Communication	Shared task queue, mutexes and condition variables (has_work, all_done) between threads of one process; pipes, shared memory between processes, message queues and semaphores are not used	Implemented (thread level)
CO4	Memory management	Dynamic task queue and inode hash table held in memory; results and statistics shared by all threads	Implemented
CO5	File system management	Recursive directory traversal, file metadata, inodes, symbolic links, hidden files, WSL mount points, history file	Implemented
CO6	Resource management	Thread count related to hardware cores; benchmark of 1, 2, 4, 8 and 12 threads; search history with runtimes	Implemented

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The program is small and does not need special hardware, but a processor with several cores is needed to observe the benefit of multithreading. Table 3.1 lists the requirements for compiling and running it.

Table 3.1: Hardware requirements

Component	Requirement
Processor	Any modern x86/x64 processor; a multi-core processor is recommended
RAM	Minimum 4 GB
Storage	At least 1 GB of free space for the project and tools
Input device	Keyboard
Output device	Display

The benchmark and search summaries in this report were captured on a system that reports 12 hardware cores (Figures 7.2 and 7.8). This is not a requirement of the program.

3.2 Software Requirements

Table 3.2: Software requirements

Software	Purpose
Ubuntu Linux / Linux environment	Operating environment that provides POSIX threads and the directory and file interfaces
GCC compiler	Compiles the C11 source code
Make	Build management, where a makefile is used
Linux terminal	Compiling, running and testing the program
POSIX threads (pthreads)	Worker threads, mutexes and condition variables
Standard C and POSIX headers	stdio, stdlib, string, ctype, time, unistd, errno, dirent, sys/stat, fnmatch and others (Table 4.2)

3.3 Operating System Environment — Ubuntu / WSL

The project runs on Linux. It can be used on a native Ubuntu installation or on Ubuntu running under the Windows Subsystem for Linux (WSL). Under WSL the Windows drives are mounted inside Linux, for example `C:` appears as `/mnt/c`, so the application can search Windows folders such as the Desktop, Downloads and Documents (Option 4). The application also converts Windows-style paths such as `C:\Users\...` into the corresponding WSL mount points such as `/mnt/c/Users/...`.

The program cannot be compiled and run natively on Windows or macOS because it depends on POSIX threads and Linux directory interfaces. Searching Windows files through `/mnt/c` can be slower than searching native Linux folders, and file names, permissions and sizes seen through the mount may differ from those of a native Linux file system.

3.4 Development Tools and Compilers

GCC (GNU Compiler Collection) is used to compile the program, with Make for build management where applicable, and the Linux terminal is used to run the compiler and the executable. The source code consists of `main.c`, which contains the command-line dashboard, and the header `search.h`, which contains the search definitions and the headers that are included (Table 4.2).

CHAPTER 4
TECHNOLOGY AND TOOLS USED

4.1 C Programming

This chapter describes the technologies and interfaces used by the application. Table 4.1 gives an overview.

Table 4.1: Technologies used

Technology	Use in the project
C11	Core systems programming language of the whole application
POSIX Threads (pthreads)	Concurrent execution of the worker threads
pthread_mutex_t	Protects shared data: task queue, statistics and output
pthread_cond_t	Condition variables has_work and all_done coordinate the workers
opendir() and readdir()	Read the entries of a directory
stat() and lstat()	File metadata; lstat() identifies symbolic links
GCC and Make	Compilation and build management, where applicable
Ubuntu Linux	Development and execution environment
Windows Subsystem for Linux	Access to Windows files through /mnt/c mount points
ANSI escape sequences	Coloured headings, paths and summaries in the terminal

C is the language of Unix and Linux, and the system-call interface and POSIX threads are defined for it. It is suitable for this project because it gives direct access to threads, directory streams and file metadata, uses explicit memory management and produces a small, fast executable without a runtime or virtual machine.

4.2 GCC Compiler

GCC translates the C source code into an executable file. Because the program uses POSIX threads, it must be compiled in C11 mode and linked with the pthread library. A typical command is shown below; the exact file names and options of the project's own build are to be confirmed from its makefile.

```
gcc -std=c11 -Wall -Wextra -pthread main.c -o <executable-name>
```

The option `-Wall` enables common warnings, `-Wextra` enables additional ones, `-pthread` enables thread support, and `-o` names the executable.

4.3 POSIX Threads and Linux APIs

The header `search.h` lists the standard and POSIX headers used by the application. Table 4.2 shows these headers and their role in the program.

Table 4.2: Headers included in search.h and their use

Header	Used for
stdio.h, stdlib.h, stdint.h, stdbool.h	Input/output, memory allocation, fixed-width integers (such as the 64-bit hash), boolean values
string.h, strings.h, ctype.h	String handling, case-insensitive comparison, character tests
time.h	Timing of searches and timestamps in the history log
limits.h, errno.h, unistd.h	Path-length limits, error reporting, POSIX system interfaces
pthread.h	POSIX threads, mutexes and condition variables
dirent.h	opendir(), readdir() and closedir() for reading directories
sys/stat.h, sys/types.h	stat(), lstat(), file size, device and inode numbers
fnmatch.h	Shell-style wildcard matching of file names

Besides these headers, the application reads file contents line by line with `getline()` in the content search, compares file contents with `memcmp()` in duplicate detection, and writes the history file with the standard file functions.

4.4 WSL Host Interoperability

WSL runs a real Linux kernel and mounts the Windows file system under `/mnt`. Option 4 uses this mount to search the Windows PC from the Linux terminal. It detects the current user's profile folder to offer the Desktop, Downloads and Documents folders (for example `/mnt/c/Users/<user>/Desktop` in Figure 7.4), the entire C: drive (`/mnt/c`) or a custom Windows path, which is translated from the `C:\...` form into the matching `/mnt/c/...` path.

4.5 Linux Commands and Utilities

Only a few shell commands are needed to build and run the project.

Table 4.3: Commands used to build and run the project

Command	Use
cd	Change to the project directory
ls	List the files in the directory (source files, executable and search_history.txt)
gcc -std=c11 -Wall -Wextra -pthread ...	Compile the source files into the executable
./<executable-name>	Run the program
cat search_history.txt	Display the history file written by the application (optional)

4.6 Other Tools

No other programming languages, frameworks, databases or external libraries are used in the project. The only stored data is the text file `search_history.txt`.

CHAPTER 5
SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a master-worker thread-pool architecture with a dynamic, non-blocking work queue (producer-consumer pattern), as shown in Figure 5.1. The command-line dashboard in `main.c` collects the user's choices and calls the orchestrator `run_search()`. The orchestrator fills the master task queue and starts the worker threads (`search_worker`). Workers process directories and files, report matches to the thread-safe results area, and a search record is finally written to `search_history.txt`.

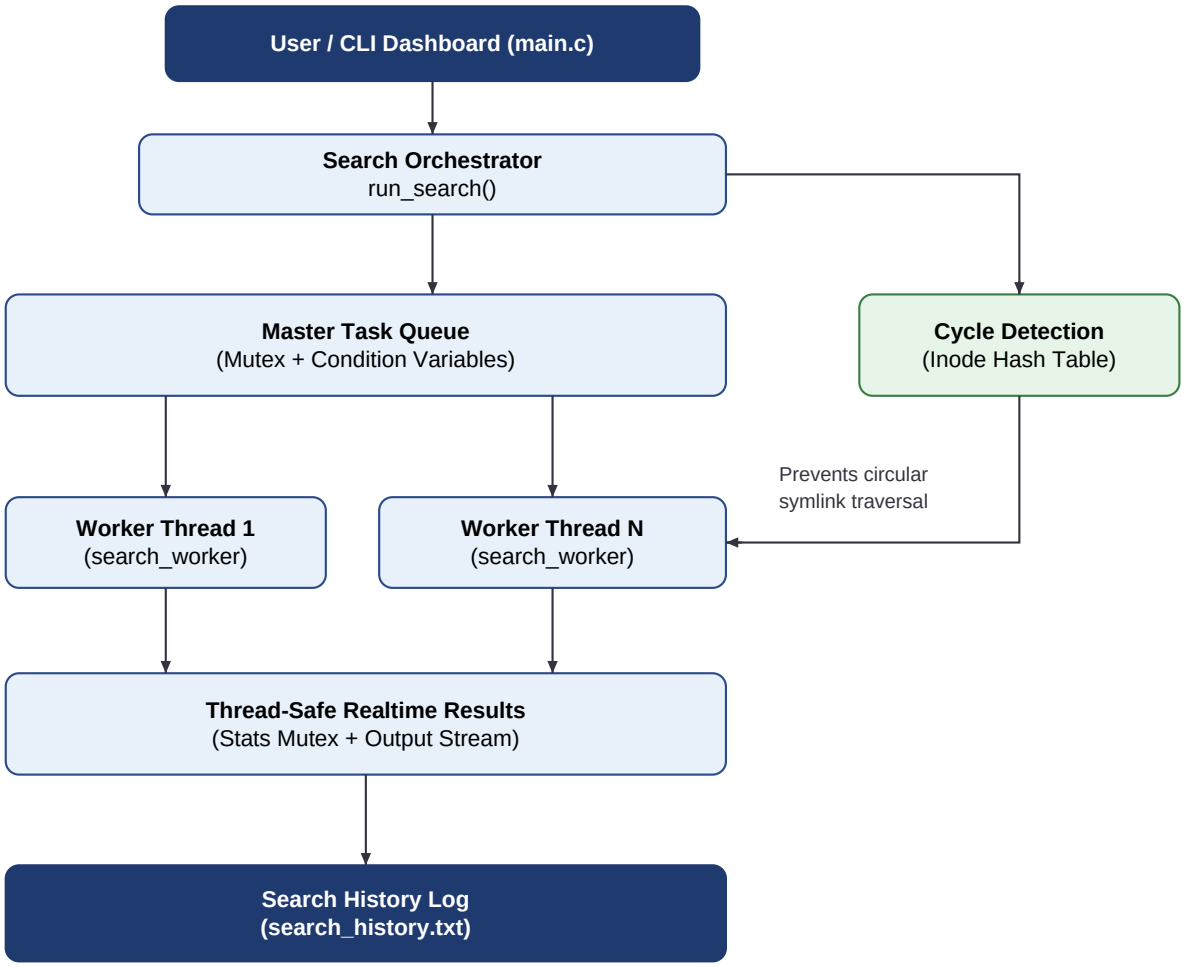


Figure 5.1: Overall system architecture

Three mechanisms make the architecture safe. **Mutexes** (`pthread_mutex_t`) protect the queue and the shared statistics and output. **Condition variables** (`pthread_cond_t`: `has_work` and `all_done`) let workers sleep until work is available and let the master wait for completion without busy-waiting. **Inode tracking** (`st_dev` and `st_ino`) prevents infinite traversal of circular symbolic links.

5.2 System Workflow

The program works in the following steps (Figure 5.2):

1. The user selects a search operation from the dashboard.
2. The application validates and normalizes the target path (Windows paths are translated to WSL mount points).
3. Search tasks are placed in a shared queue.
4. Worker threads process directories and files concurrently.
5. Results and statistics are updated safely under mutex protection.
6. Results are displayed and the search history is recorded.

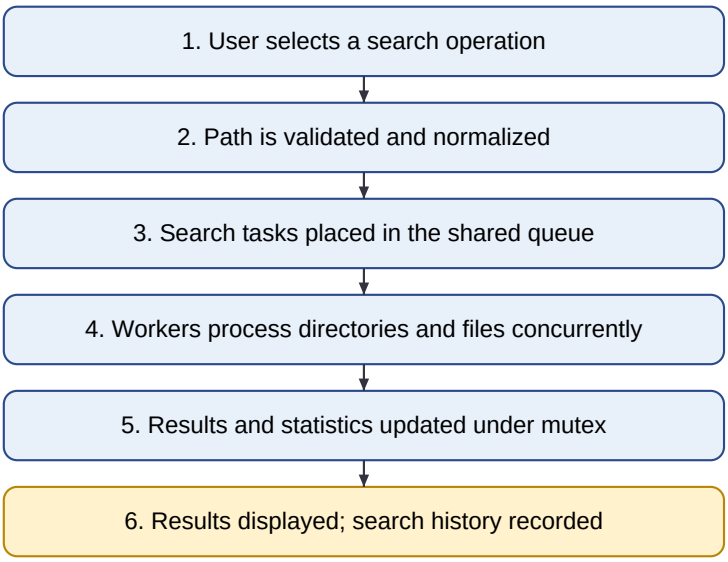


Figure 5.2: System workflow

The following safeguards are applied during the workflow: permission errors are handled so that an unreadable folder does not stop the search; binary files are detected and skipped during text searches; symbolic-link cycles are detected using inode tracking; and the worker threads are terminated properly when the work is finished.

5.3 Flowchart / Data Flow

Figure 5.3 shows the flowchart of the main program. After the dashboard is displayed, the user enters a number from 1 to 10. Option 10 prints a goodbye message and ends the program; every other option runs the selected operation, displays the result, and waits for the user to press Enter before the dashboard is shown again.

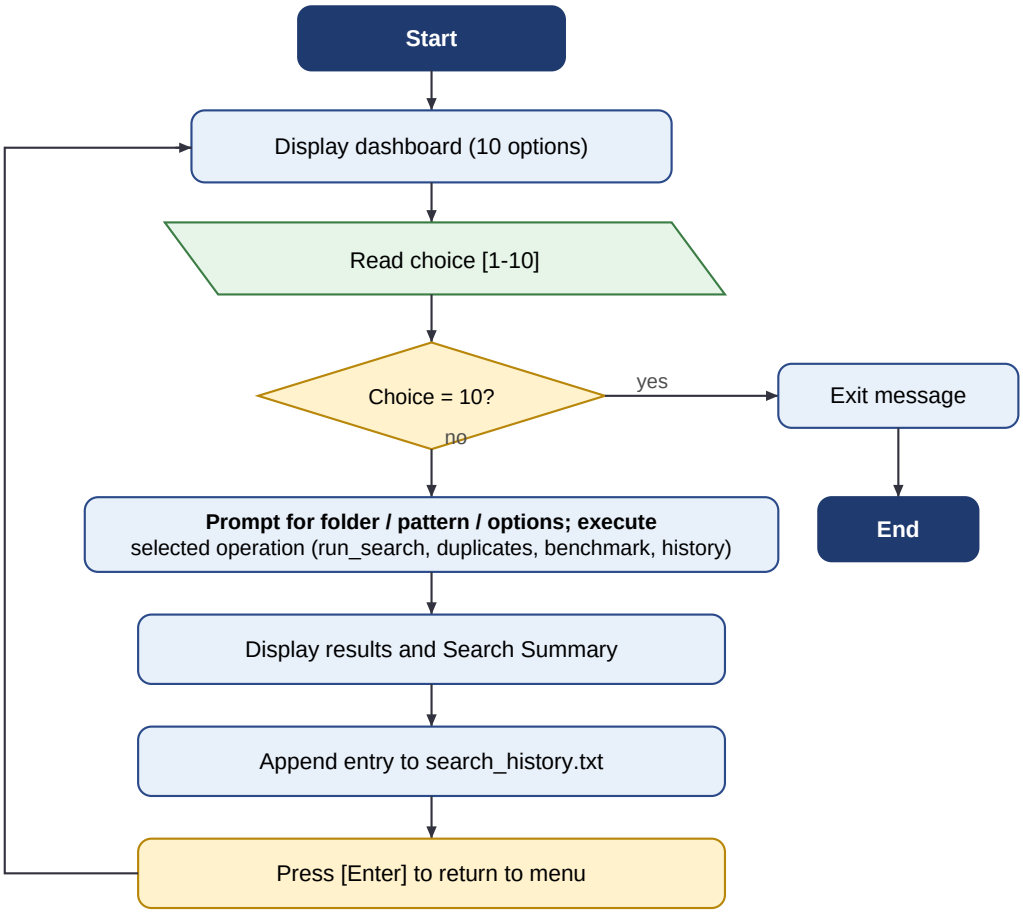


Figure 5.3: Main program flowchart

5.4 Module Description

Table 5.1 describes each option of the main menu, the mechanism it uses and what it displays.

Table 5.1: Module description

No.	Module	Purpose	Mechanism	Output
1	Search by Filename	Find files and folders by name	Case-insensitive substring or shell wildcard (*.c, test*, report_??.pdf)	Paths with sizes; folders marked Directory; Search Summary
2	Search Text Inside Files	Find text in file contents	Line-by-line scan with getline(); binary files skipped	Path, line number and code preview; Search Summary
3	Search by File Extension	Group files by type	Matches .pdf or pdf, .c or *.c	Matching paths; Search Summary
4	Search Windows PC	Search Windows files from WSL	/mnt/c mount points; filename or text search	Matching Windows paths and sizes
5	Search by File Size	Find large or small files	Threshold >= or <= in Bytes, KB, MB or GB	Matching files; Search Summary
6	Search Hidden / Config Files	Find dotfiles and hidden folders	Names starting with a dot; optional name filter	Matching paths; Search Summary
7	Find Duplicate Files	Detect redundant copies	Size match, FNV-1a hash, memcmp	Duplicate sets, extra copies, reclaimable space
8	Benchmark Performance	Measure speedup	Runs with 1, 2, 4, 8, 12 threads; T1 / TN	Benchmark matrix table
9	View Search History Log	Review previous searches	Reads search_history.txt; c clears it	Timestamp, type, folder, query, threads, time, matches
10	Exit	End the program	Safe termination	Goodbye message

5.5 Input and Output Design

Input. All input is given from the keyboard. The user enters a menu number from 1 to 10, followed by the prompts of the selected option. Most prompts show a default value in square brackets that is used when the user presses Enter, for example the folder `[.]`, the filename pattern `[*]`, the Windows location `[1]` or the size unit `[1]`.

Output. Output is shown in the terminal with coloured headings, matching paths (directories are shown in a different colour) and a Search Summary. The summary shows the files scanned, the matches found, the threads used together with the number of hardware cores, the time elapsed and the scanning speed in files per second. The history module also produces file output: each search is appended to `search_history.txt`. The screens are shown in Chapter 7.

Table 5.2: Module input and output

Module	Input from user	Output
Option 1	Folder; filename or pattern	Matching files and directories; Search Summary
Option 2	Folder; text to find	Path:line: preview; Search Summary
Option 3	Folder; extension	Matching files; Search Summary
Option 4	Location 1–5; search type 1 or 2; search term	Matching Windows paths
Option 5	Folder; filter 1 or 2; size number; unit 1–4	Files above or below the threshold; Search Summary
Option 6	Folder; optional name filter	Hidden files and folders; Search Summary
Option 7	Folder	Duplicate File Sets; files checked; extra copies; time
Option 8	Folder to benchmark	Multithreaded Benchmark Matrix
Option 9	c to clear the log or Enter to return	Search History Log

CHAPTER 6
IMPLEMENTATION

6.1 Module-wise Implementation

This chapter describes the implementation of the application as it is described in the project architecture and observed in its output. The program consists of the source file `main.c`, which contains the dashboard and the menu loop, and the header `search.h`. The search logic is shared by all options and is started through the orchestrator `run_search()`.

6.1.1 Master-Worker Thread Pool and Task Queue

`run_search()` creates the master task queue, places the starting folder in it and creates the worker threads (`search_worker`). By default the number of workers equals the number of hardware cores; the test system reports 12 cores and 12 active worker threads (Figure 7.2). A task is a directory or file waiting to be examined. A worker removes a task from the queue; if it is a directory, the worker reads it with `opendir()` and `readdir()`, examines each entry with `lstat()`, and puts subdirectories back into the queue as new tasks. Because every free worker takes the next task, the load is distributed dynamically.

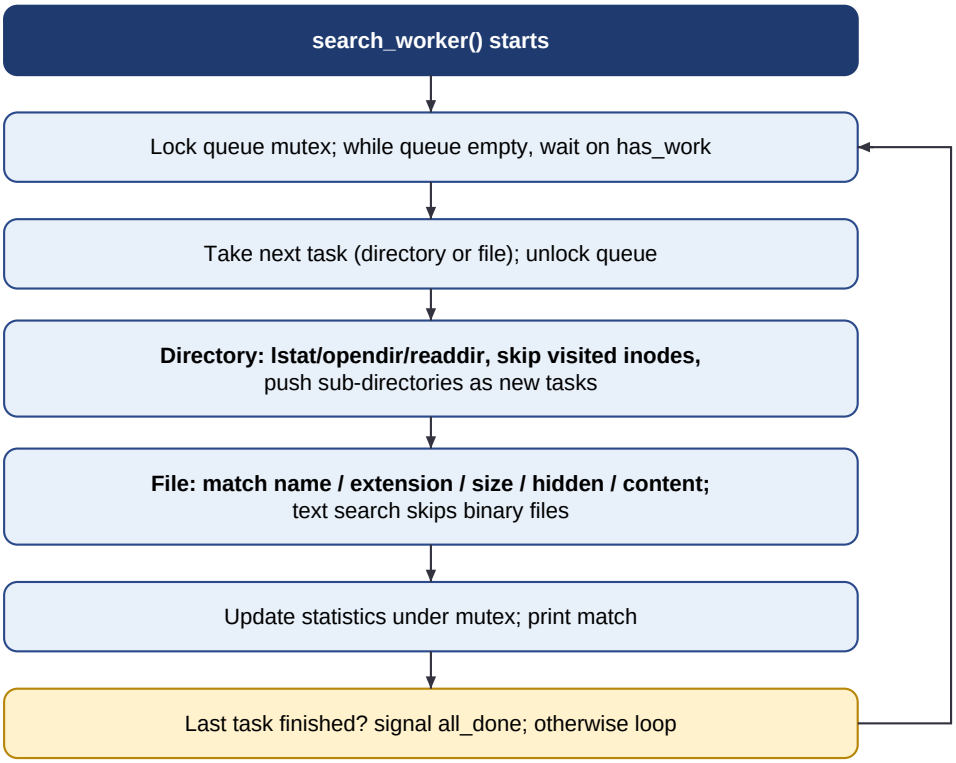


Figure 6.1: Flow of a worker thread (search_worker)

6.1.2 Synchronization

The queue is protected by a mutex. When the queue is empty, a worker waits on the condition variable `has_work` instead of repeatedly checking the queue, so no processor time is wasted. When a task is added, a waiting worker is woken. The master waits on `all_done`, which is signalled when all tasks have been processed, after which the workers are stopped properly. The counters of the search statistics (files scanned and matches found) and the output stream are protected by a separate stats mutex, so that results appear in real time without being mixed together.

6.1.3 Cycle Detection Using Inodes

A symbolic link can point to a parent folder and make a directory tree contain a cycle. The program therefore uses `lstat()` to recognise symbolic links and keeps an inode hash table of the directories that have been visited. Each entry is identified by the pair `st_dev` and `st_ino`; a directory whose pair is already in the table is not entered again, which prevents circular symbolic-link traversal.

6.1.4 Binary File Detection and Error Handling

During text searches the program inspects the content of a file with a heuristic: a file that contains null bytes, or in which more than 15 % of the characters are control characters, is treated as binary or executable and is skipped. Permission errors and other filesystem errors, such as a folder that cannot be opened, are handled by skipping the item so that the rest of the search continues.

6.1.5 Option 1 — Search by Filename

The user enters a folder and a filename or pattern. The match is case-insensitive substring matching, or shell wildcard matching when the pattern contains wildcards such as `*.c`, `test*` or `report_??.pdf`. Both files and folder names are matched, so folders are listed with the label (Directory). The search is recursive. Results show the path and file size, followed by the Search Summary (Figure 7.2).

6.1.6 Option 2 — Search Text Inside Files

The workers open ASCII/UTF-8 files and read them line by line with `getline()`. For every line that contains the text, the file path, the line number and a truncated preview of the line are printed immediately. Binary files are skipped (Section 6.1.4). The option is useful for finding function definitions, TODO comments, API keys or error messages in a code base (Figure 7.3).

6.1.7 Option 3 — Search by File Extension

The program matches files whose names end with the given extension. The extension can be entered in several forms: `.pdf` and `pdf`, or `.c` and `*.c`, are treated alike. The option groups files by type, for example all source files, spreadsheets, images or configuration files of a project.

6.1.8 Option 4 — Search Windows PC

The option uses the WSL mount `/mnt/c` to search the Windows file system from Linux. It detects the current user's profile directory and offers: 1 Desktop, 2 Downloads, 3 Documents, 4 Entire C: drive and 5 Custom Windows path. The user then chooses the search type, 1 for filename or 2 for text inside files, and enters the search term. A Windows path of the form `C:\...` is translated into `/mnt/c/...` (Figure 7.4).

6.1.9 Option 5 — Search by File Size

The user enters a folder, chooses a filter, 1 for larger than or 2 for smaller than a threshold, enters a number and selects a unit: 1 MB, 2 KB, 3 GB or 4 Bytes. The program finds files that are larger than or equal to (>=) or smaller than or equal to (<=) the threshold. It is used for disk clean-up, for example to find large videos, ISO files, big logs or zero-byte files (Figure 7.5).

6.1.10 Option 6 — Search Hidden / Config Files

The option scans dotfiles and hidden folders, such as `.env`, `.bashrc`, `.git` and `.vscode`. An optional name filter such as `env` or `git` narrows the result; pressing Enter lists all hidden files (Figure 7.6).

6.1.11 Option 7 — Find Duplicate Files

Duplicate detection uses a three-stage filter so that expensive comparisons are done only when necessary (Figure 6.2). First, files are grouped by size, because files of different sizes cannot be identical. Second, files of equal size are compared by a 64-bit FNV-1a hash; the hash step reduces the number of unnecessary comparisons. Third, files whose hashes match are verified byte by byte with `memcmp()`, which confirms that their content is really identical. The result lists the duplicate sets with the original path and the duplicate paths, the number of extra copies, and the total reclaimable storage space (Figure 7.7).



Figure 6.2: Duplicate-file verification pipeline

6.1.12 Option 8 — Benchmark Performance

The benchmark runs the same search on the chosen folder with 1, 2, 4, 8 and 12 threads and measures the time of each run. The speedup is calculated as the ratio of the single-thread time to the time with N threads:

```
Speedup(N) = T1 / TN        (T1 = time with 1 thread, TN = time with N threads)
```

The result is shown as a Multithreaded Benchmark Matrix with the thread count, time in seconds, files checked and speedup (Figure 7.8). The result depends on the workload, the storage and caching; a large folder such as `/usr/include` gives more meaningful results than a folder of a few files.

6.1.13 Option 9 — View Search History Log

After each search the application appends a line to `search_history.txt` containing the timestamp, the search type (Filename, Content, Windows-PC, Duplicates and so on), the folder, the query, the number of threads, the runtime and the number of matches. Option 9 reads and displays this file. Pressing `c` clears the log and pressing Enter returns to the dashboard (Figure 7.10).

6.1.14 Option 10 — Exit

The option terminates the program safely and displays a goodbye message.

6.2 Algorithm / Pseudocode

The pseudocode below describes the main algorithms of the application.

Algorithm 1: Parallel search (run_search)

```
READ folder, search mode and search term from the user
VALIDATE and normalize the path ; IF it cannot be accessed THEN report error and RETU
RN
INITIALIZE task queue, queue mutex, stats mutex, condition variables has_work and all
_done, inode table
PUT starting folder in the task queue
CREATE worker threads
WAIT on all_done until every task has been processed
STOP workers and JOIN them
DISPLAY results and Search Summary ; APPEND a record to search_history.txt
DESTROY mutexes and condition variables and release memory
```

Algorithm 2: Worker thread (search_worker)

```
LOOP
  LOCK queue mutex
  WHILE queue is empty AND work is not finished: WAIT on has_work
  IF finished THEN UNLOCK and EXIT
  TAKE next task ; UNLOCK queue mutex
  IF task is a directory THEN
     IF (st_dev, st_ino) is already in the inode table THEN SKIP
     ELSE record it ; OPEN directory (skip on permission error)
          FOR each entry: lstat() ; ADD sub-directories to the queue as tasks ;
                          apply the match test to files and directory names
  ELSE apply the match test to the file
  LOCK stats mutex ; update counters ; print match ; UNLOCK
  IF no tasks remain and no worker is busy THEN SIGNAL all_done
```

Algorithm 3: Binary-file detection

```
READ the beginning of the file
IF a null byte is found THEN RETURN binary
IF control characters are more than 15 % of the characters THEN RETURN binary
RETURN text
```

Algorithm 4: Duplicate-file detection

```
COLLECT all regular files with their sizes
GROUP files by size ; DISCARD groups with one file
FOR each remaining file: COMPUTE 64-bit FNV-1a hash
GROUP files of the same size by hash ; DISCARD groups with one file
FOR each remaining group: COMPARE files byte by byte with memcmp()
REPORT sets of identical files, extra copies and reclaimable space
```

Algorithm 5: Benchmark (speedup matrix)

```
FOR N in {1, 2, 4, 8, 12}: run the search with N threads ; measure time TN
speedup(N) <- T1 / TN
DISPLAY threads, time, files checked and speedup
```

6.3 Core C Implementation

Table 6.1 lists the main parts of the implementation as they are known from the project architecture. The source code listing itself is not reproduced in this report.

Table 6.1: Main parts of the implementation

Part	Role
main.c	Command-line dashboard: menu loop, prompts with default values and display of results
search.h	Declarations, constants and the standard and POSIX headers used by the search engine (Table 4.2)
run_search()	Orchestrator: validates the path, fills the task queue, starts and joins the workers, displays the summary
search_worker	Thread function executed by every worker
Master task queue	Shared queue of directory and file tasks protected by a mutex and the condition variable has_work
Condition variable all_done	Signals the master that all tasks have been completed
Stats mutex and output stream	Thread-safe counters and real-time printing of results
Inode hash table	Device and inode pairs of visited directories for cycle detection
search_history.txt	Text file with one record per search

[INSERT SELECTED SOURCE CODE EXTRACTS FROM main.c AND search.h HERE, FOR EXAMPLE THE WORKER THREAD LOOP AND THE DUPLICATE CHECK]

6.4 Linux System Calls / APIs Used

Table 6.2: Linux system calls and APIs used

Function / API	Purpose	Module
pthread_create(), pthread_join()	Create the worker threads and wait for them	All searches, benchmark
pthread_mutex_t	Protect the queue, statistics and output	All searches
pthread_cond_t (has_work, all_done)	Wake workers; signal completion	All searches
opendir(), readdir(), closedir()	Read directory entries	All searches
stat(), lstat()	File size, type, device and inode; detect symbolic links	All searches
fnmatch()	Shell wildcard matching of names	Options 1, 4
getline()	Read a file line by line	Options 2, 4
memcmp()	Byte-by-byte verification of duplicate candidates	Option 7
time functions	Measure the runtime and write timestamps	All searches, history
Standard file I/O	Write and read search_history.txt	Option 9

6.5 Process / Thread Implementation

The application is a single process that creates a pool of POSIX threads for every search. The number of workers is the number of hardware cores by default (12 on the test system), and the benchmark runs the same search with 1, 2, 4, 8 and 12 threads. The workers share the memory of the process, which allows them to use one task queue and one set of statistics. Workers are joined and the resources are released when the search is complete. The program does not create child processes.

6.6 IPC / Synchronization Implementation

The workers communicate through shared memory inside one process: the task queue and the results. Access is synchronized with mutexes (`pthread_mutex_t`), and the workers and the master are coordinated with the condition variables `has_work` and `all_done`, which avoid busy-waiting. The inode table prevents the same directory from being processed twice. Inter-process mechanisms such as pipes between processes, message queues and semaphores are not used.

6.7 Memory Management Implementation

Memory management in the project consists of managing the data structures of the search. The task queue and the inode hash table are dynamic data structures that grow as the directory tree is explored, and the results and statistics are shared by all threads. The project does not implement a custom memory manager. Memory is released and the synchronization objects are destroyed when a search ends. Large file contents are not loaded as a whole: text files are read line by line.

6.8 User Interface Implementation

The user interface is a text-based dashboard. A coloured title **PARALLEL FILE SEARCH** is shown above the numbered menu of ten options and the prompt `Choose an option [1-10]` (Figure 7.1). Each option begins with a coloured heading such as `--- 1. Search by Filename ---` and asks for its input with default values in square brackets. During a search, the line `Starting search with 12 threads in '.'...` is shown, matching paths are printed in real time, and a Search Summary closes the output. After the output, the message `Press [Enter] to return to menu...` is displayed. The colours are produced with ANSI escape sequences, so the terminal must support them.

CHAPTER 7
EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

This chapter explains how the program is compiled and run, and demonstrates each module with screenshots captured from the running application, which are included without modification. Open the Ubuntu terminal, change to the directory that contains the source files and compile with GCC; the exact command of the project's build is to be confirmed from its makefile.

```
cd <project-directory>
gcc -std=c11 -Wall -Wextra -pthread main.c -o <executable-name>
```

7.2 Execution Commands

```
./<executable-name>
```

The program should be run in a terminal that supports ANSI colours. The history file `search_history.txt` is created in the directory from which the program is started.

7.3 Main Menu

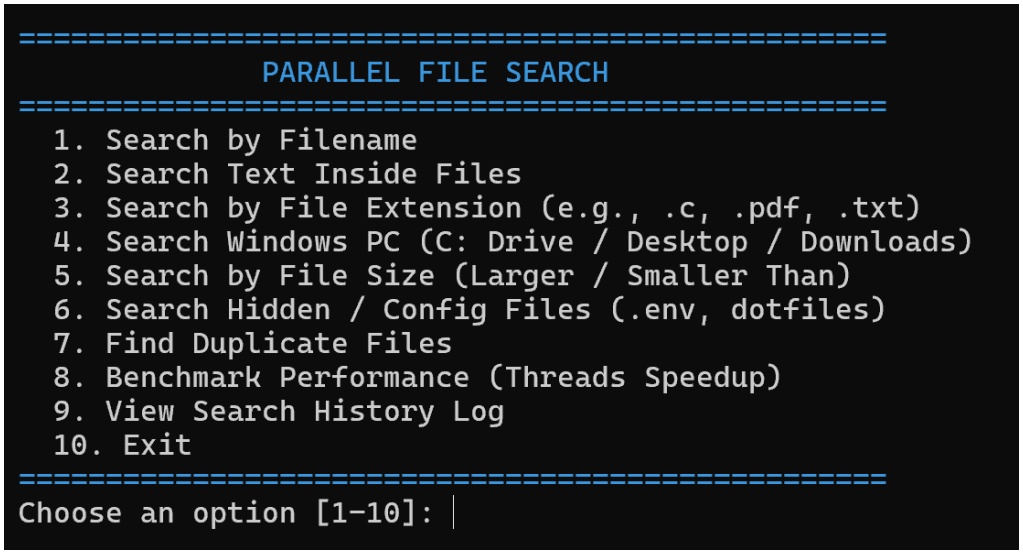


Figure 7.1: Main menu of the Parallel File Search application

Figure 7.1 shows the dashboard with the title PARALLEL FILE SEARCH and ten options: 1 Search by Filename, 2 Search Text Inside Files, 3 Search by File Extension, 4 Search Windows PC, 5 Search by File Size, 6 Search Hidden / Config Files, 7 Find Duplicate Files, 8 Benchmark Performance, 9 View Search History Log and 10 Exit. The user enters the number of the required option.

7.4 Search by Filename

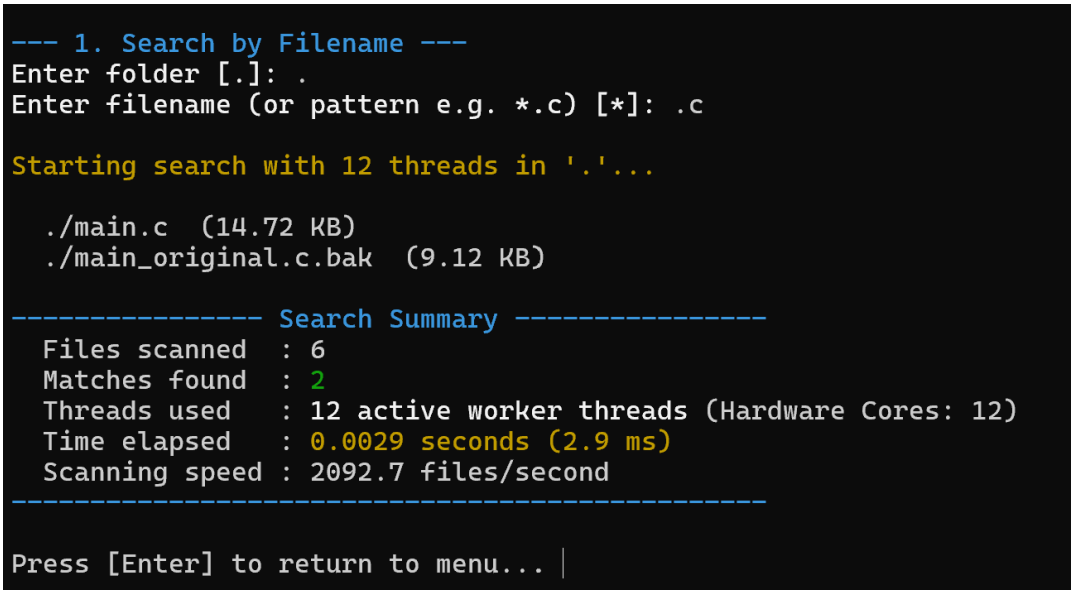


Figure 7.2: Search by filename (Option 1)

Figure 7.2 shows a filename search in the current folder with the pattern `.c`. The program reports `Starting search with 12 threads` and lists two matches, `./main.c` (14.72 KB) and `./main_original.c.bak` (9.12 KB). The Search Summary shows 6 files scanned, 2 matches found, 12 active worker threads (hardware cores: 12), a time of 0.0029 seconds (2.9 ms) and a scanning speed of 2092.7 files per second.

7.5 Search Text Inside Files

```
--- 2. Search Text Inside Files ---
Enter folder [.]: .
Enter text to find: #include

Scanning text with 12 threads in '.'...

  ./main.c:1: #include "search.h"
  ./main_original.c.bak:3: #include <stdio.h>
  ./main_original.c.bak:4: #include <stdlib.h>
  ./main_original.c.bak:5: #include <string.h>
  ./main_original.c.bak:6: #include <pthread.h>
  ./main_original.c.bak:7: #include <dirent.h>
  ./main_original.c.bak:8: #include <sys/stat.h>
  ./main_original.c.bak:9: #include <limits.h>
  ./main_original.c.bak:10: #include <unistd.h>
  ./main_original.c.bak:11: #include <errno.h>
  ./search.h:5: #include <stdio.h>
  ./search.h:6: #include <stdlib.h>
  ./search.h:7: #include <stdint.h>
  ./search.h:8: #include <stdbool.h>
  ./search.h:9: #include <string.h>
  ./search.h:10: #include <strings.h>
  ./search.h:11: #include <ctype.h>
  ./search.h:12: #include <time.h>
  ./search.h:13: #include <limits.h>
  ./search.h:14: #include <unistd.h>
  ./search.h:15: #include <errno.h>
  ./search.h:16: #include <pthread.h>
  ./search.h:17: #include <dirent.h>
  ./search.h:18: #include <sys/stat.h>
  ./search.h:19: #include <sys/types.h>
  ./search.h:20: #include <fnmatch.h>
  ./search_history.txt:2: [2026-10-16 14:02:45] Type: Content

---------------- Search Summary ----------------
  Files scanned  : 6
  Matches found  : 27
  Threads used   : 12 active worker threads (Hardware Cores: 12)
  Time elapsed   : 0.0041 seconds (4.1 ms)
  Scanning speed : 1461.1 files/second
```

Figure 7.3: Search text inside files (Option 2)

Figure 7.3 shows a content search for `#include` in the current folder. The program prints each matching file, line number and preview, for example `./main.c:1:` followed by `#include "search.h"`, and also matches in `main_original.c.bak` and `search.h`. The Search Summary shows 6 files scanned, 27 matches found, 12 worker threads, a time of 0.0041 seconds (4.1 ms) and a scanning speed of 1461.1 files per second.

7.6 Search by File Extension

[INSERT SCREENSHOT OF OPTION 3 — SEARCH BY FILE EXTENSION, FOR EXAMPLE EXTENSION .c OR json]

Option 3 asks for a folder and an extension. The extension may be entered as `.c` or `c`. The matching files are printed, followed by the Search Summary. The screenshot is to be added after this option has been executed.

7.7 Search Windows PC

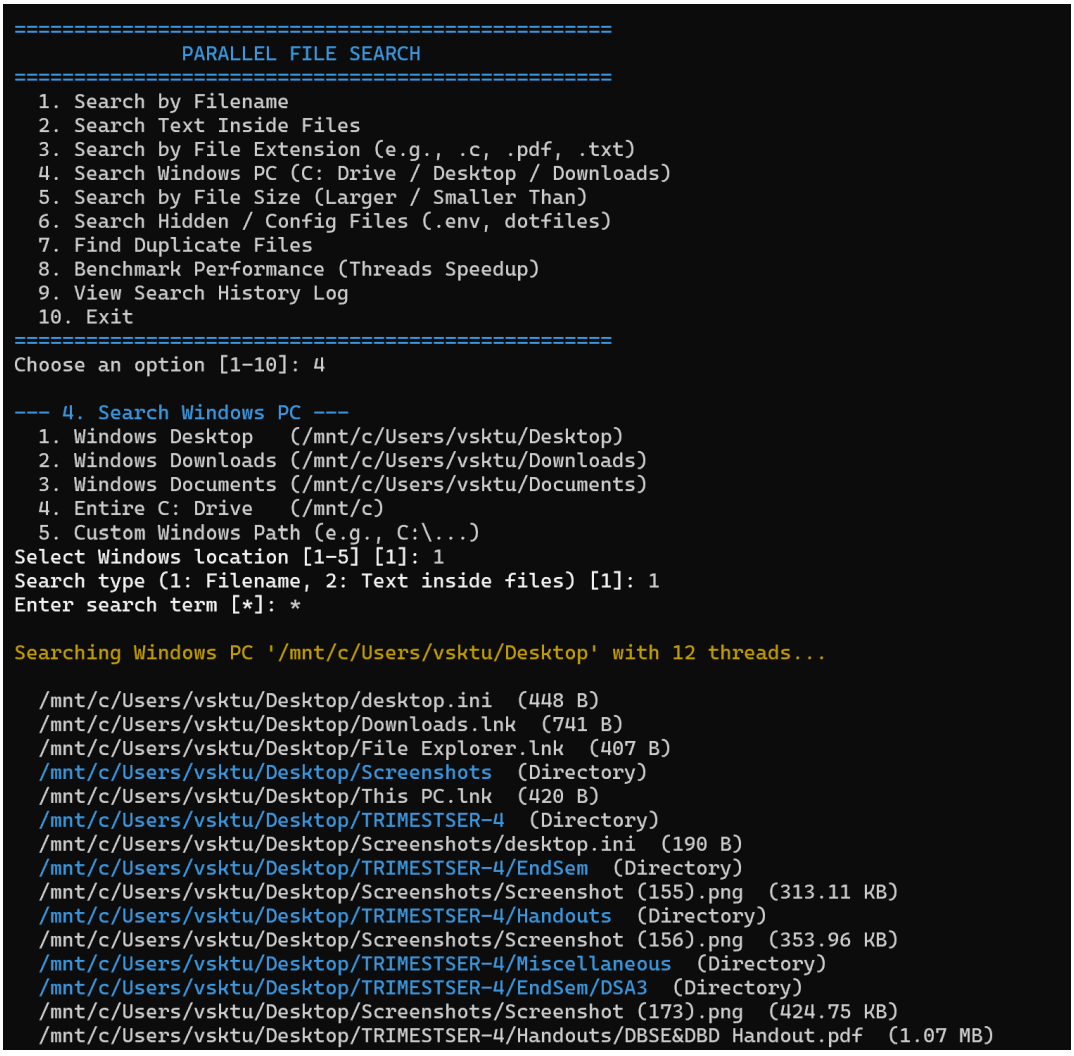


Figure 7.4: Search Windows PC through WSL (Option 4)

Figure 7.4 shows Option 4. The five quick locations (Desktop, Downloads, Documents, entire C: drive and custom Windows path) are listed with their WSL paths under `/mnt/c/Users/...`. Location 1 (Desktop), search type 1 (filename) and the term `*` were selected, and the program reports `Searching Windows PC ... with 12 threads`. The output lists Windows files with

their sizes and folders marked (Directory). The history log (Figure 7.10) records this search with 122 matches in 0.1950 seconds.

7.8 Search by File Size

```
--- 5. Search by File Size ---
Enter folder [.]: .
  1. Larger than threshold  (e.g., > 10 MB)
  2. Smaller than threshold (e.g., < 100 KB)
Choose filter [1-2] [1]: 1
Enter size number (e.g. 10, 50, 500) [10]: 50
  1. MB  |  2. KB  |  3. GB  |  4. Bytes
Choose unit [1-4] [1]: 1

Scanning for files larger than 50.00 MB in '.' (12 threads)...


---------------- Search Summary ----------------
  Files scanned  : 6
  Matches found  : 0
  Threads used   : 12 active worker threads (Hardware Cores: 12)
  Time elapsed   : 0.0039 seconds (3.9 ms)
  Scanning speed : 1525.5 files/second
```

Figure 7.5: Search by file size (Option 5)

Figure 7.5 shows a search for files larger than 50 MB in the current folder, using filter 1 and unit 1 (MB). The message `Scanning for files larger than 50.00 MB in '.' (12 threads)` is displayed. The Search Summary reports 6 files scanned and 0 matches found, because no file in the folder is that large, in 0.0039 seconds.

7.9 Search Hidden / Config Files

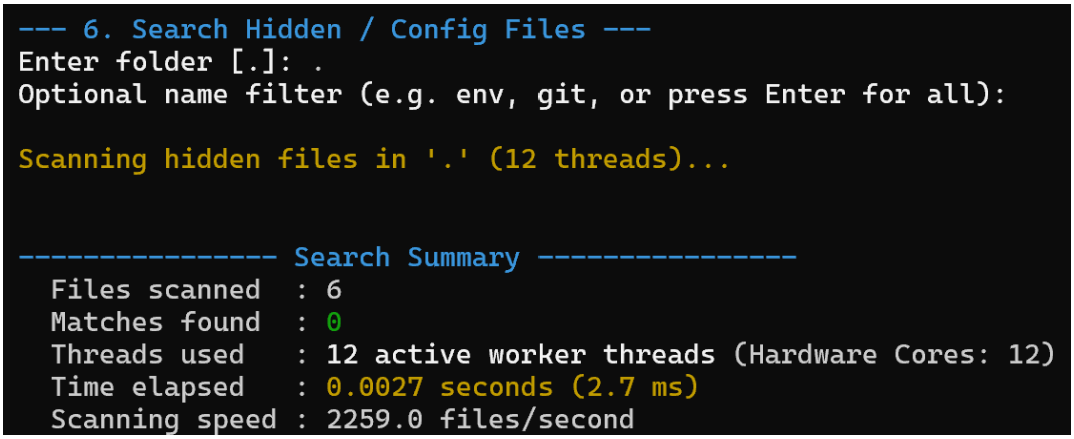


Figure 7.6: Search hidden and configuration files (Option 6)

Figure 7.6 shows Option 6 with no name filter. The search of the current folder scanned 6 files and found 0 hidden files in 0.0027 seconds, because the test folder contains no dotfiles.

7.10 Find Duplicate Files

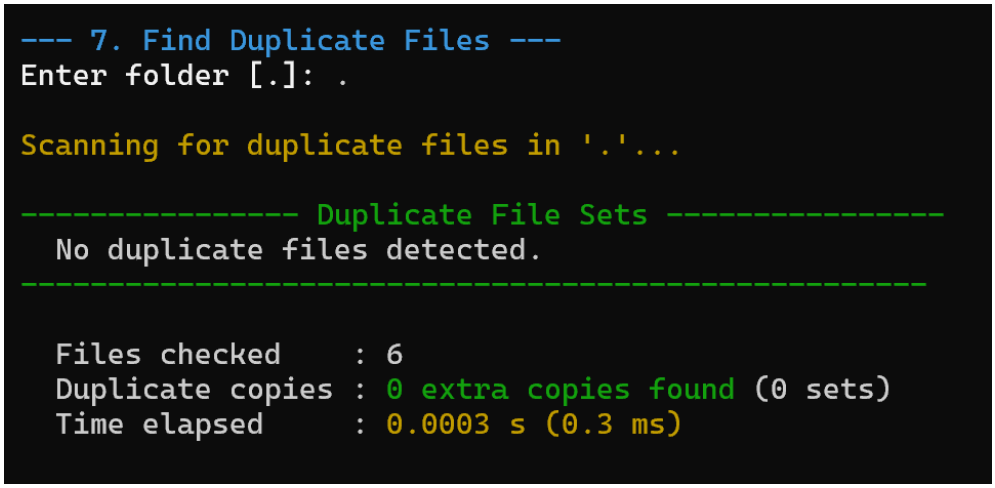


Figure 7.7: Find duplicate files (Option 7)

Figure 7.7 shows the duplicate scan of the current folder. The program reports `No duplicate files detected.`; 6 files were checked, 0 extra copies were found (0 sets) and the time elapsed was 0.0003 seconds. In a folder that contains duplicates the screen lists the duplicate sets and the reclaimable storage (Section 6.1.11).

7.11 Benchmark Performance

```
--- 8. Benchmark Performance (Threads Speedup) ---
Enter folder to benchmark [.]: .

================================================
        MULTITHREADED BENCHMARK MATRIX
  Target: .                          Hardware: 12 Cores
================================================

  Threads      Time (sec)      Files Checked      Speedup
  ------------------------------------------------------
  1             0.0013 s         6                 1.00x
  2             0.0008 s         6                 1.60x
  4             0.0012 s         6                 1.07x
  8             0.0031 s         6                 0.42x
  12            0.0029 s         6                 0.46x
```

Figure 7.8: Multithreaded benchmark matrix (Option 8)

Table 7.1: Benchmark results on the test folder (6 files, 12 hardware cores)

Threads	Time (sec)	Files checked	Speedup (T1 / TN)
1	0.0013	6	1.00x
2	0.0008	6	1.60x
4	0.0012	6	1.07x
8	0.0031	6	0.42x
12	0.0029	6	0.46x

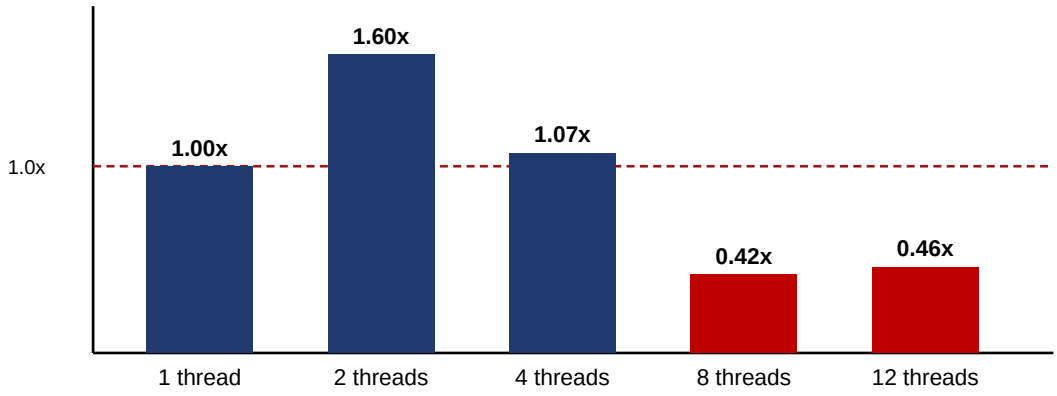


Figure 7.9: Speedup measured for different thread counts (values from Figure 7.8)

Observations: the test folder contains only 6 files, so each search takes only a few milliseconds. With 2 threads the search was 1.60 times faster than with one thread, but with 8 and 12 threads the speedup fell below 1.0x (0.42x and 0.46x), because the time needed to create and coordinate threads is larger than the work they share. This result agrees with the note that more threads do not always give a faster search. The test folder is too small to show the benefit of many threads;

the benchmark should be repeated on a large directory such as `/usr/include`, and the measurements recorded in this report. [ADD LARGE-FOLDER BENCHMARK RESULT AND SCREENSHOT AFTER EXECUTION]

7.12 Search History Log

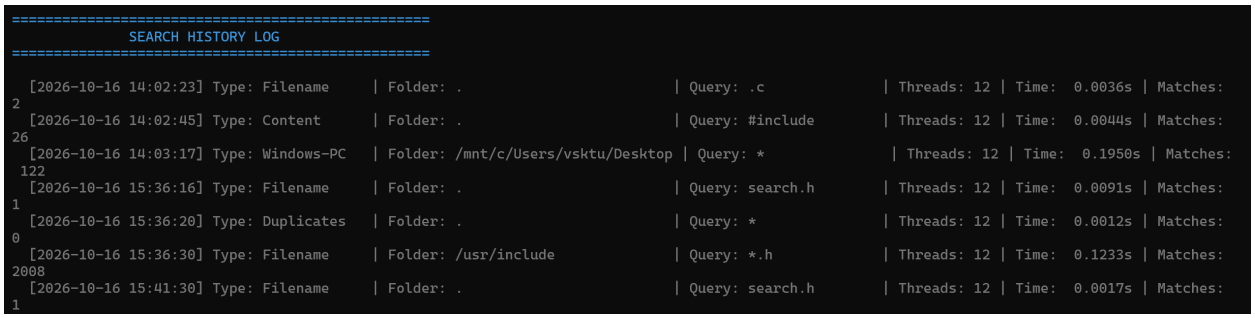


Figure 7.10: Search history log (Option 9)

Table 7.2: Records shown in Figure 7.10 (transcribed from the screenshot)

Timestamp	Type	Folder	Query	Threads	Time	Matches
2026-10-16 14:02:23	Filename	.	.c	12	0.0036s	2
2026-10-16 14:02:45	Content	.	#include	12	0.0044s	26
2026-10-16 14:03:17	Windows-PC	/mnt/c/Users/vsktu/Desktop	*	12	0.1950s	122
2026-10-16 15:36:16	Filename	.	search.h	12	0.0091s	1
2026-10-16 15:36:20	Duplicates	.	*	12	0.0012s	0
2026-10-16 15:36:30	Filename	/usr/include	*.h	12	0.1233s	2008
2026-10-16 15:41:30	Filename	.	search.h	12	0.0017s	1

Figure 7.10 shows the history log with seven records. Each record contains the timestamp, search type, folder, query, number of threads, time and matches: for example a Filename search of `.` for `.c` (2 matches), a Content search for `#include`, a Windows-PC search of `/mnt/c/Users/vsktu/Desktop` for `*` (122 matches in 0.1950 s), a Duplicates search (0 matches), and a Filename search of `/usr/include` for `*.h`, which found 2008 matches in 0.1233 seconds with 12 threads. The logged match count of the content search (26) differs by one from the 27 matches shown on screen in Figure 7.3; the team should verify the reason. [VERIFY]

7.13 Testing

The program was tested by running the menu options and checking that the expected screen appears. Table 7.3 lists the test cases. In the column Actual Result, only cases that are supported by the screenshots in this report are marked as verified. The remaining cases are to be completed by the team after executing them; no results have been assumed.

Table 7.3: Test cases and execution results

Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC01	Start application	./<executable>	Menu with 10 options is displayed	Verified (Figure 7.1)	Pass
TC02	Filename search	Option 1, folder ., pattern .c	Matching files and Search Summary	2 matches, 6 files scanned (Figure 7.2)	Pass
TC03	Content search	Option 2, text #include	Paths, line numbers, previews	27 matches (Figure 7.3)	Pass
TC04	Extension search	Option 3, extension .c	Matching files	[Record after execution]	[Pass / Fail]
TC05	Windows PC search	Option 4, Desktop, filename *	Windows files listed	Files and folders listed (Figure 7.4); 122 matches logged	Pass
TC06	Size search	Option 5, > 50 MB	Files larger than 50 MB or none	0 matches (Figure 7.5)	Pass
TC07	Hidden files search	Option 6, no filter	Hidden files or none	0 matches (Figure 7.6)	Pass
TC08	Duplicate detection	Option 7, folder .	Duplicate sets or none	No duplicates, 6 files (Figure 7.7)	Pass
TC09	Benchmark	Option 8, folder .	Speedup matrix for 1, 2, 4, 8, 12 threads	Matrix displayed (Figure 7.8)	Pass
TC10	Search history	Option 9	Records of previous searches	Seven records (Figure 7.10)	Pass
TC11	Clear history log	Option 9, c	Log is cleared	[Record after execution]	[Pass / Fail]
TC12	Duplicate files present	Folder with copies	Duplicate sets and reclaimable space	[Record after execution]	[Pass / Fail]
TC13	Binary file skipped	Option 2 on binary file	Binary file is not searched	[Record after execution]	[Pass / Fail]
TC14	Symbolic-link cycle	Folder with circular link	Search terminates	[Record after execution]	[Pass / Fail]
TC15	Exit operation	Option 10	Goodbye message; program ends	[Record after execution]	[Pass / Fail]

7.14 Demonstration of OS Concepts

Table 7.4: Demonstration of OS concepts

Feature	OS Concept	Implementation
Thread pool	Process and thread management	POSIX worker threads created for every search; 12 workers on the test system
Task queue	Producer-consumer problem	Workers consume tasks and add sub-directories as new tasks
Mutex locks	Mutual exclusion, race conditions	pthread_mutex_t protects queue, statistics and output
Condition variables	Thread synchronization	has_work and all_done avoid busy-waiting
Directory traversal	File system management	opendir(), readdir(), lstat()
Inode tracking	Inodes and symbolic links	st_dev and st_ino of visited directories in a hash table
Windows PC search	File system interoperability	WSL mount /mnt/c, path translation
Duplicate detection	File content and data integrity	Size match, FNV-1a hash, memcmp verification
Benchmark	Performance and resource management	T1 / TN speedup for 1, 2, 4, 8, 12 threads
History log	File handling	Records appended to search_history.txt

CHAPTER 8
CONCLUSION

8.1 Summary of the Project

Parallel File Search Using Threads is a menu-driven terminal application written in C11 with POSIX threads for Linux/Ubuntu, including Ubuntu running under WSL. It distributes search work among a pool of worker threads through a shared task queue and provides ten menu options: filename search, content search, extension search, Windows PC search, size search, hidden-file search, duplicate detection, a performance benchmark, a search history log and exit.

During the project, the team applied concepts of the Operating Systems course to a real program: threads and thread pools, the producer-consumer problem, mutexes and condition variables, directory traversal, inodes and symbolic links, and performance measurement. The program is intended as an educational application and not as a replacement for professional search tools.

8.2 Objectives Achieved

Table 8.1: Objectives and implemented results

No.	Objective (Section 1.4)	Implemented result
1	File-search application in C11	Application with ten menu options (Figure 7.1)
2	Multithreading with POSIX threads	Master-worker thread pool (Section 6.1.1)
3	Filename searches	Option 1 (Section 6.1.5)
4	Text search with binary detection	Option 2 and binary heuristic (Sections 6.1.4, 6.1.6)
5	Metadata searches and Windows search	Options 3 to 6 (Sections 6.1.7 to 6.1.10)
6	Recursive traversal and dynamic distribution	Shared task queue (Section 6.1.1)
7	Mutexes and condition variables	Section 6.1.2
8	Safe filesystem handling	Permission-error handling and inode tracking (Sections 6.1.3, 6.1.4)
9	Duplicate detection	Option 7 (Section 6.1.11)
10	Performance measurement and history	Options 8 and 9 (Sections 6.1.12, 6.1.13)
11	Understand OS concepts	Concept mapping in Tables 2.2 and 7.4

8.3 Results Achieved

The application starts from a single command and gives access to all search operations from one menu. The screenshots in this report show that the filename, content, Windows PC, size, hidden-file, duplicate and benchmark options and the history log produce the expected output. Searches of the small test folder (6 files) finished in about 3 to 4 milliseconds, the Windows Desktop search with 122 matches took 0.1950 seconds, and a search of `/usr/include` for `*.h` found 2008 matches in 0.1233 seconds (Figure 7.10). The benchmark on the small test folder showed a speedup of 1.60x with 2 threads and below 1.0x with 8 and 12 threads, which shows that the benefit of threads depends on the workload. No further performance claims are made in this report.

8.4 Skills and Learning Outcomes

- C programming: structures, pointers, dynamic data structures, string handling and hashing.
- POSIX thread programming: thread creation, mutexes, condition variables and thread pools.
- Concurrency: race conditions, producer-consumer coordination and avoiding busy-waiting.
- Linux file systems: directory traversal, file metadata, inodes and symbolic links.
- WSL: accessing Windows files through Linux mount points.
- Performance measurement: timing, speedup calculation and interpreting benchmark results.
- GCC compilation and the Linux terminal.
- Teamwork: dividing the work among team members and integrating it into one program.

8.5 Limitations

- The interface is terminal-based; there is no graphical dashboard.
- The implementation is specific to Linux and POSIX threads.
- Searches are case-insensitive; regular expressions are not supported.
- Performance depends on storage speed, file count, file sizes and caching; creating too many threads can increase overhead, as the benchmark of a small folder shows.
- Files and folders that cannot be accessed are skipped and cannot be searched.
- Content search is limited to ASCII/UTF-8 text; binary files are skipped by a heuristic that may occasionally misclassify a file.
- The history log is a plain text file without filtering options.

- Values obtained under WSL, especially when searching Windows drives, can differ from those on native Linux.

8.6 Future Scope

The following improvements are possible in future versions. They are not features of the current application.

- A case-sensitive search option and regular-expression or other advanced pattern matching.
- More advanced search filters, for example size ranges and modification dates.
- A progress display while the workers are processing tasks.
- Improved visualization of results, including a graphical user interface.
- Optimization for very large directory trees.
- Additional benchmark analysis, including comparison on larger folders and different storage devices.
- Improved error reporting that lists inaccessible directories and unreadable files.

CHAPTER 9
REFERENCES

9.1 Books

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2014.

[3] M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.

[4] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013.

[5] D. R. Butenhof, Programming with POSIX Threads, Addison-Wesley, 1997.

[6] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

9.2 Research Papers

No research papers were used directly in the implementation of this project. [ADD ANY PAPERS THAT THE TEAM ACTUALLY READ AND CITED, OR LEAVE THIS STATEMENT.]

9.3 Linux / GNU Documentation

The following documentation describes the interfaces used by the application. Last accessed: [ACCESS DATE].

[7] Linux man-pages project, pthreads(7). https://man7.org/linux/man-pages/man7/pthreads.7.html

[8] Linux man-pages project, pthread_create(3). https://man7.org/linux/man-pages/man3/pthread_create.3.html

[9] Linux man-pages project, pthread_mutex_lock(3p). https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html

[10] Linux man-pages project, pthread_cond_wait(3p). https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html

[11] Linux man-pages project, opendir(3) and readdir(3). https://man7.org/linux/man-pages/man3/opendir.3.html

[12] Linux man-pages project, stat(2) and lstat(2).
https://man7.org/linux/man-pages/man2/stat.2.html

[13] Linux man-pages project, fnmatch(3).
https://man7.org/linux/man-pages/man3/fnmatch.3.html

[14] Linux man-pages project, getline(3). https://man7.org/linux/man-pages/man3/getline.3.html

[15] Linux man-pages project, inode(7) and symlink(7).
https://man7.org/linux/man-pages/man7/inode.7.html

[16] GCC, the GNU Compiler Collection – Online Documentation.
https://gcc.gnu.org/onlinedocs/

[17] The GNU C Library Reference Manual. https://www.gnu.org/software/libc/manual/

9.4 Online Resources

[18] The Open Group Base Specifications Issue 7 (POSIX.1-2008).
https://pubs.opengroup.org/onlinepubs/9699919799/

[19] Microsoft, Working across Windows and Linux file systems (WSL).
https://learn.microsoft.com/en-us/windows/wsl/filesystems

[20] G. Fowler, L. C. Noll and K.-P. Vo, FNV Hash.
http://www.isthe.com/chongo/tech/comp/fnv/

[21] Project repository: [ADD REPOSITORY LINK]